Text To SQL Documentation

Title: Text To SQL Documentation
Date: 05/05/2025
Prepared by: Sidhanth Kamath
Reviewed by: Suraj Nagraj
Version: v5.0

Contents

1. Overview 2
2. Login & Authentication 2
3. Main Application Components 5
4. Frontend Implementation Details 6
5. Backend Integration with the Main Application 13
6. Component Interactions 18
7.Cost Analysis 19
8. Appendix 19

1. Overview

The Text-to-SQL Assistant is an interactive web application that enables users to generate SQL queries using natural language processing. This enterprise-grade application features secure authentication, an intuitive chat interface, database connectivity options, and robust visualization capabilities. The system is designed for users who need to query databases without extensive SQL knowledge.

2. Login & Authentication

2.1 Authentication Architecture

The application implements a secure authentication flow using Key cloak, an open-source Identity and Access Management solution. The authentication is structured in three main components:

a. **Key cloak Utility (keycloak.ts)**: Core authentication functions

b. **Authentication Context (KeycloakProvider.tsx)**: React context management for auth state

c**. Login UI (login.tsx)**: User interface for the authentication process

2.2 Key Components

a. **Key cloak Utility Layer**

The `keycloak.ts` file implements core authentication functions:

- Configuration: Connects to Key cloak using environment variables
- Authentication: Manages user login with silent SSO capability
- Session Management: Handles token storage and retrieval
- Logout: Securely terminates user sessions

b. **Authentication Context Provider**

The `KeycloakProvider.tsx` implements a React Context that:

- Manages authentication state across the application
- Provides login/logout functionality to components
- Handles automatic redirects based on auth status
- Synchronizes authentication state across browser tabs
- Sets up API authorization headers for authenticated requests

c. **Login Interface**

The `login.tsx` component provides:

- User-friendly login interface with corporate branding
- Visual feedback during authentication initialization
- Automatic redirects for authenticated users
- Responsive design for various device sizes

d. **Silent SSO Check**

The `silent-check-sso.html` file is a critical component for seamless authentication:

- Purpose: Enables background authentication verification without disrupting user experience
- Implementation: A minimal HTML page that receives and relays authentication responses
- Function: Works as a communication bridge between Key cloak server and the application
- Process:
  1. Receives authentication response via redirect from Key cloak
  2. Posts the authentication information to the parent application window
  3. Allows the main application to update session state without page reload

2.3 How Silent SSO Works

a. The Key cloak client periodically checks if the user's session is still valid

b. It loads `silent-check-sso.html` in a hidden iframe

c. The Key cloak server redirects to this page with authentication status

d. The script posts the complete URL (containing tokens or error information) to the parent window

e. The parent application processes this information to maintain session state

f. This entire process happens invisibly to the user, enabling a seamless experience

2.4 User Experience

a. **Initial Access**: Users navigate to the application and see the login screen with EG branding

b. **Authentication Process**: Users are redirected to Key cloak to authenticate using their EG credentials

c. **Authorization**: Upon successful authentication, users return to the application with appropriate access rights

d. **Session Management**: Users remain authenticated until logout or session expiration

e. **Seamless Session Refresh**: The silent-check-SSO mechanism refreshes tokens without user intervention

f. **Crosstab Synchronization**: Authentication state is synchronized across browser tabs

2.5 Security Features

- PKCE (Proof Key for Code Exchange) authentication for enhanced security
- Automatic token refresh to maintain sessions
- Secure token storage in memory (not in local storage)
- Silent authentication verification without page reloads
- Crosstab session synchronization
- Comprehensive error handling for authentication failures (As shown in Fig 1.)

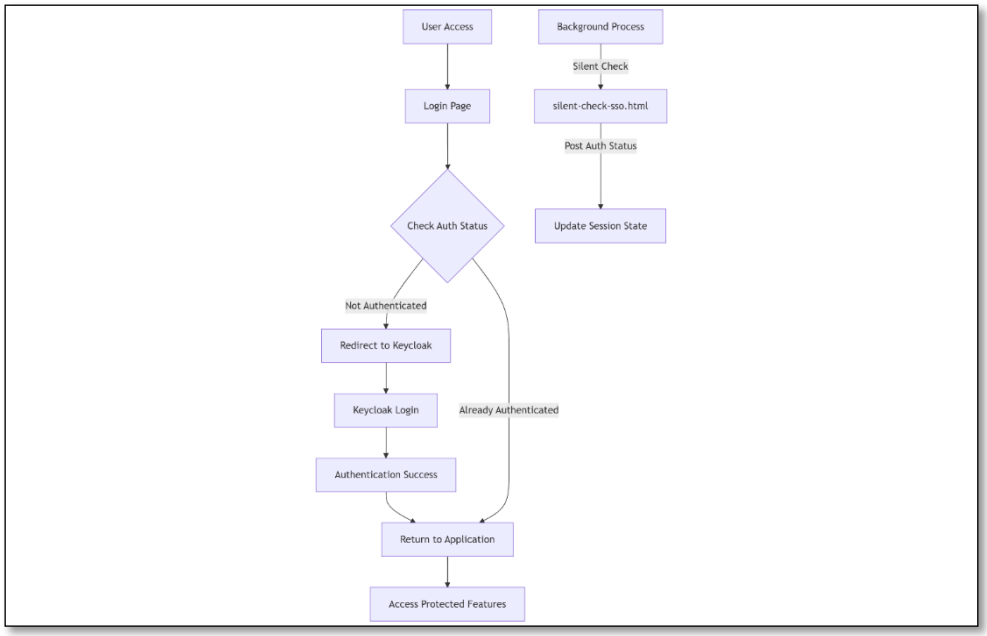
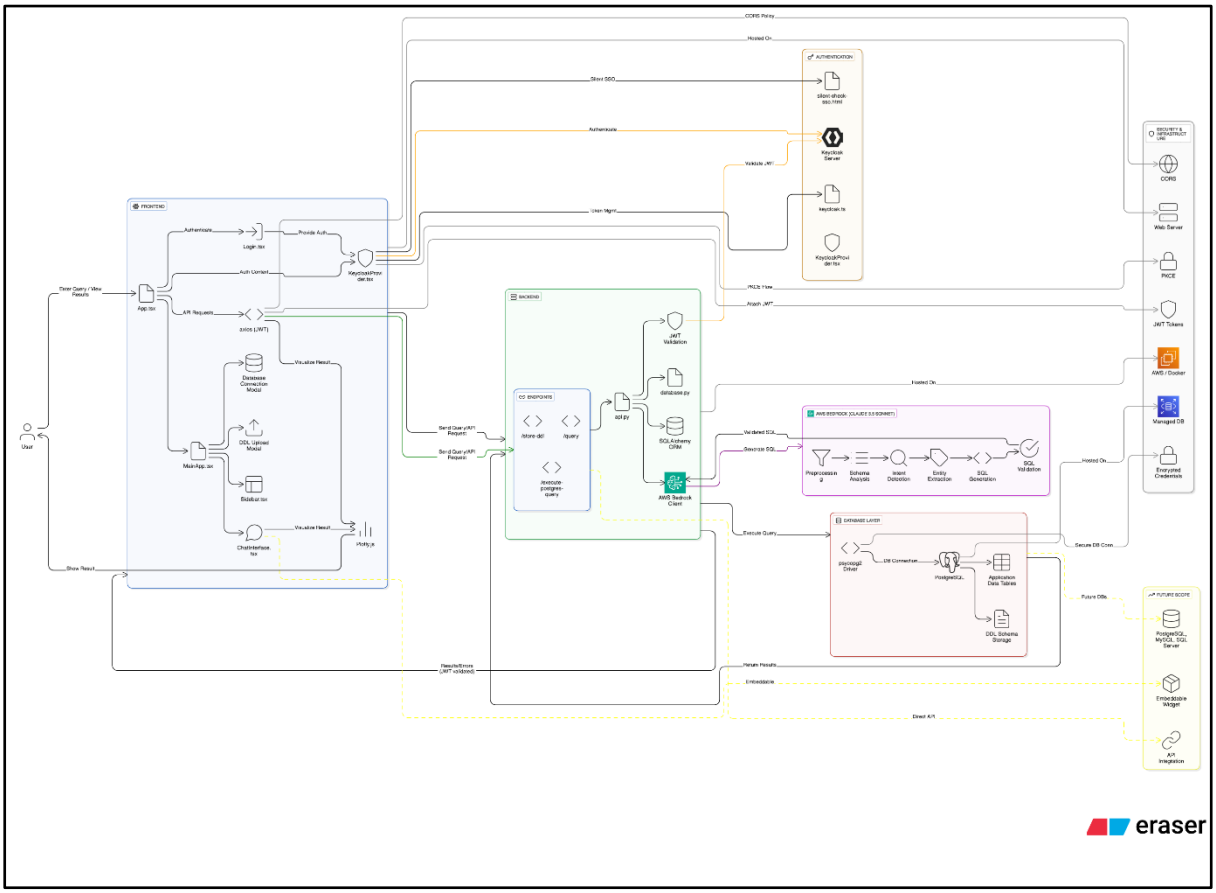


Fig 1. Authentication Flow

This architecture ensures a secure, user-friendly authentication experience while protecting the Text-to-SQL Assistant application from unauthorized access.

3. Main Application Components



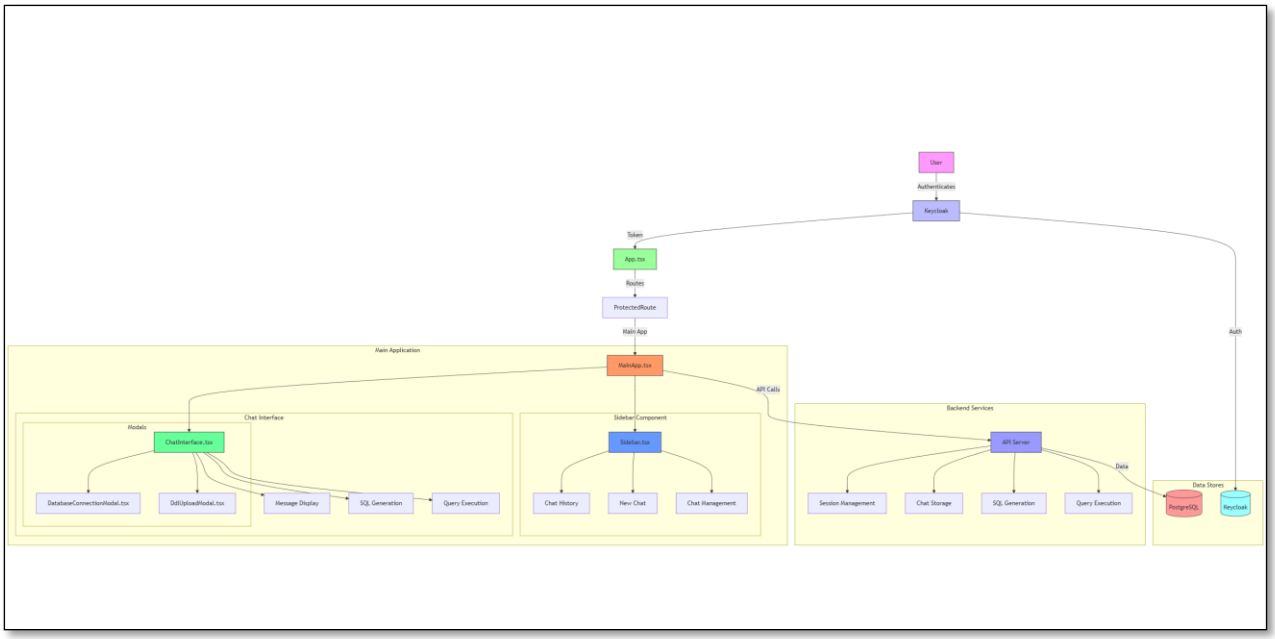


Fig 2. System Architecture

3.1 Components

- Frontend: React-based user interface with chat interface and visualization tools
- Backend: Python-based API service with natural language processing capabilities
- Database Connector: Interface for connecting to various database systems
- NLP Engine: Processes natural language input and generates SQL queries (As shown in Fig 2.)

3.2 Data Flow

- User inputs a natural language question
- Backend processes the question using NLP techniques
- System generates SQL query
- Query is executed against the connected database
- Results are returned and displayed to the user (As shown in Fig 2.)

4. Frontend Implementation Details

4.1 Technology Stack

- **Framework**: React with TypeScript
- **State Management**: React Context API for global state

- **Styling**: CSS modules with Tailwind CSS
- **Build Tool**: Vite
- **Authentication**: Keycloak integration
- **Data Visualization**: Plotly.js

4.2 Component Structure

Main Components

- **App.tsx**: Entry point of the application, manages routing and global providers
- **MainApp.tsx**: Primary layout component containing sidebar and main content area
- **ChatInterface.tsx**: Manages conversation flow and message display
- **Sidebar.tsx**: Navigation component that manages chat history display and selection

Context Providers

- **DatabaseContext**: Manages database connection state and methods
- **DdlContext**: Handles DDL file management and schema information
- **KeycloakProvider**: For Authentication

Modal Components

- **DatabaseConnectionModal**: Interface for configuring database connections
- **DdlUploadModal**: Allows users to upload database schema definitions

4.3 API Integration

```
import axios from "axios";
import { getKeycloak } from "./keycloak";
import { DatabaseConnection } from '../context/DatabaseContext';

// Add these interfaces at the top of the file
interface UserAccessResponse {
  access_granted: boolean;
  business_unit?: string;
  is_admin?: boolean;
```

```
  email?: string;
  message?: string;
}

interface DdlResponse {
  status: string;
  message?: string;
  ddl_id: string;
  file_name?: string;
  content_length?: number;
  parse_status?: 'pending' | 'completed' | 'error';
  tables?: string[];
  table_columns?: Record<string, string[]>;
  schema_info?: string;
  error_message?: string;
}

interface PostgresQueryResponse {
  results: Array<{ connection_test: number }>;
  status: string;
  message?: string;
}

interface DatabaseTestResponse {
  success: boolean;
  data: PostgresQueryResponse;
}

const getBaseURL = () => {
  // Check if window is available (client-side)
  if (typeof window === 'undefined') {
    // Return a default value or from environment variable for server-side
    return process.env.VITE_BACKEND_URL || '/api';
  }

  const host = window.location.host;
  const protocol = window.location.protocol;

  // Special handling for localhost development
  if (host.includes('localhost') || host.includes('127.0.0.1')) {
    // You may need to adjust the port based on your backend configuration
    const port = '8000'; // Change this to your backend port if different
    return `${protocol}//${host.split(':')[0]}:${port}`;
  }
  // console.log('host', host);
  // console.log('protocol', protocol);
  // console.log('full URL', `${protocol}//api.${host}`);
  // For all domains and subdomains
  return `${protocol}//api.${host}`;
}
```

```
const axiosPrivateInstance = axios.create({
  // baseURL: import.meta.env.VITE_BACKEND_URL ,
  baseURL: getBaseURL(),
  headers: {
    'Content-Type': 'application/json'
  },
  withCredentials: true
});

// Add request interceptor with token validation
axiosPrivateInstance.interceptors.request.use(
  (config) => {
    const token = sessionStorage.getItem('keycloakToken');
    if (token && config.headers) {
      config.headers.Authorization = `Bearer ${token}`;
      if (!config.headers['Content-Type'] && !(config.data instanceof FormData)) {
        config.headers['Content-Type'] = 'application/json';
      }
    }
    return config;
  },
  (error) => {
    console.error('Request interceptor error:', error);
    return Promise.reject(error);
  }
);

// Add isRefreshing flag to prevent multiple simultaneous refresh attempts
let isRefreshing = false;
let refreshSubscribers: ((token: string) => void)[] = [];

const addRefreshSubscriber = (callback: (token: string) => void) => {
  refreshSubscribers.push(callback);
};

const onRefreshed = (token: string) => {
  refreshSubscribers.forEach((callback) => callback(token));
  refreshSubscribers = [];
};

// Add response interceptor to handle authorization errors
axiosPrivateInstance.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;

    // Add specific handling for access revoked
    if (error.response?.status === 403 &&
        error.response?.data?.detail?.includes("inactive")) {
```

```
      // console.log("User access has been revoked");
      sessionStorage.clear();
      localStorage.clear();
      window.location.href = '/login';
      return Promise.reject(error);
    }

    if ((error.response?.status === 403 || error.response?.status === 401) &&
!originalRequest._retry) {
      if (isRefreshing) {
        return new Promise((resolve) => {
          addRefreshSubscriber((token: string) => {
            if (originalRequest.headers) {
              originalRequest.headers.Authorization = `Bearer ${token}`;
            }
            resolve(axiosPrivateInstance(originalRequest));
          });
        });
      }

      originalRequest._retry = true;
      isRefreshing = true;

      try {
        const kc = getKeycloak();
        const refreshed = await kc.updateToken(70);

        if (refreshed && kc.token) {
          const newToken = kc.token;
          sessionStorage.setItem('keycloakToken', newToken);
          axiosPrivateInstance.defaults.headers.common['Authorization'] = `Bearer
${newToken}`;
          onRefreshed(newToken);
          return axiosPrivateInstance(originalRequest);
        }
      } catch (refreshError) {
        console.error('Token refresh failed:', refreshError);
        sessionStorage.clear();
        localStorage.clear();
        window.location.href = '/login';
      } finally {
        isRefreshing = false;
      }
    }
    return Promise.reject(error);
  }
);

export const uploadDdlFile = async (file: File, sessionId: string):
Promise<DdlResponse> => {
```

```
  try {
    const formData = new FormData();
    formData.append('ddl_file', file);
    formData.append('session_id', sessionId);

    const response = await axiosPrivateInstance.post<DdlResponse>("/store-ddl",
formData, {
      headers: {
        'Content-Type': 'multipart/form-data'
      }
    });
    return response.data;
  } catch (error) {
    console.error('Error uploading DDL file:', error);
    throw error;
  }
};

export const submitDdlText = async (ddlText: string, sessionId: string):
Promise<DdlResponse> => {
  try {
    const formData = new FormData();
    formData.append('ddl_text', ddlText);
    formData.append('session_id', sessionId);

    const response = await axiosPrivateInstance.post<DdlResponse>("/store-ddl",
formData, {
      headers: {
        'Content-Type': 'multipart/form-data'
      }
    });
    return response.data;
  } catch (error) {
    console.error('Error submitting DDL text:', error);
    throw error;
  }
};

// Add function to load existing DDL content
export const getDdlContent = async (ddlId: string): Promise<DdlResponse> => {
  try {
    const response = await axiosPrivateInstance.get<DdlResponse>(`/ddl-
content/${ddlId}`);
    return response.data;
  } catch (error) {
    console.error('Error fetching DDL content:', error);
    throw error;
  }
};
```

```
export const verifyUserAccess = async (email: string): Promise<boolean> => {
  try {
    const response = await axiosPrivateInstance.post<UserAccessResponse>("/users/check-access", { email });
    return response.data.access_granted;
  } catch (error) {
    console.error("Access verification failed:", error);
    return false;
  }
};


// Update the testDatabaseConnection function
export const testDatabaseConnection = async (params: DatabaseConnection): Promise<DatabaseTestResponse> => {
  try {
    const response = await axiosPrivateInstance.post<PostgresQueryResponse>(
      '/execute-postgres-query',
      {
        ...params,
        query: 'SELECT 1 as connection_test'
      }
    );

    return {
      success: true,
      data: response.data
    };
  } catch (error) {
    console.error('Database connection test failed:', error);
    throw error;
  }
};

export { axiosPrivateInstance };
```

Authentication Flow

- User login initiates Keycloak authentication
- Upon successful authentication, access token is stored
- API requests include token in Authorization header
- Token refresh is handled automatically
- Session timeout redirects to login

Chat Interface Implementation

- Maintains message history with unique IDs
- Manages loading states during query processing
- Supports markdown rendering for formatted responses

- Implements auto-scroll behaviour for new messages
- DDL Schema Upload
- DB Connection
- Highlighted keywords-based query
- Execution of query

Visualization Features

- Automatic chart type selection based on data structure
- Interactive controls for adjusting visualization parameters
- Export functionality for charts and data tables
- Responsive design for various screen sizes

5. Backend Integration with the Main Application

5.1 Overview

The Text-To-SQL application follows a client-server architecture with the following key components (as shown in the Fig 3.):

- Fast API Backend: Core server handling SQL generation and database operations
- PostgreSQL Database: Stores application data (users, chats, sessions)
- Keycloak: Authentication and authorization provider
- AWS Bedrock: LLM service for natural language to SQL conversion
- React Frontend: User interface (communicates with backend via API client)

```
+---------------+          +-----------------+
|               |          |                 |
|  React        |<--------->|  FastAPI        |<--------> AWS Bedrock (Claude LLM)
|  Frontend     |   API    |  Backend        |
|               |          |                 |
+---------------+          +-----------------+
                                   ^
                                   |
                                   v
          +------------+    +----------------+    +------------+
          |            |    |                |    |            |
          | Keycloak   |<-->| PostgreSQL DB  |<-->| User's DB  |
          |            |    | (App Data)     |    | (For SQL   |
          +------------+    +----------------+    |  Execution)|
                                                  +------------+
```

Fig 3. Front End/Back End Interaction

5.2 Technology Stack

- **Framework**: FastAPI
- **Database ORM**: SQLAlchemy
- **NLP Engine**: Custom implementation with language model integration
- **Authentication**: JWT validation with Keycloak integration
- **Database Drivers**: psycopg2 (PostgreSQL), pymysql (MySQL), pyodbc (SQL Server)

5.3 API Endpoints

Query Processing

```
# API.py
@app.post("/query")
async def process_query(
    request: QueryRequest,
    current_user: User = Depends(get_current_user)
):
    """
    Process natural language query and return results

    Parameters:
    - question: Natural language question
    - connection_id: ID of the database connection to use

    Returns:
    - SQL query generated
    - Result data
```

```
    - Execution metadata
    """
    try:
        # Get connection details from database
        connection = get_connection(request.connection_id, current_user.id)

        # Process natural language to SQL
        sql_query, confidence = nlp_processor.generate_sql(
            request.question,
            connection.schema
        )

        # Execute query against database
        results, column_names = database.execute_query(
            connection.connection_string,
            sql_query
        )

        # Format and return results
        return {
            "sql": sql_query,
            "data": results,
            "columns": column_names,
            "confidence": confidence,
            "execution_time": execution_time
        }
    except Exception as e:
        logger.error(f"Query processing error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))
```

5.4 Database Connection Management

```
# database.py
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session
from dotenv import load_dotenv
import os

# Load environment variables from .env file
load_dotenv()

# Get database connection details from environment variables
DB_HOST = os.getenv("DB_HOST")
DB_PORT = os.getenv("DB_PORT")  # Default PostgreSQL port
DB_USER = os.getenv("DB_USER")
DB_PASSWORD = os.getenv("DB_PASSWORD")
DB_NAME = os.getenv("DB_NAME")
```

```
# Validate required environment variables
missing_vars = []
for var_name in ["DB_HOST", "DB_PORT", "DB_USER", "DB_PASSWORD",
"DB_NAME"]:
    if not locals()[var_name]:
        missing_vars.append(var_name)

if missing_vars:
    raise EnvironmentError(f"Missing required environment variables: {',
'.join(missing_vars)}")


# Construct PostgreSQL connection string
DATABASE_URL =
f"postgresql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_
NAME}"

# Create SQLAlchemy engine
engine = create_engine(DATABASE_URL)

# Session factory
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# Base class for models
Base = declarative_base()

# Database session dependency
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

5.5 NLP Processing Pipeline

- **Preprocessing**: Clean and normalize input text
- **Schema Analysis**: Parse database schema for entity recognition
- **Intent Classification**: Determine query type and objectives
- **Entity Extraction**: Identify tables, columns, conditions from text
- **SQL Generation**: Construct valid SQL based on extracted information
- **Validation**: Verify SQL syntax and security constraints
- **Execution**: Run query against target database

5.6 Database Schema Management

- Automatic schema extraction from database connections
- Manual schema definition through DDL file uploads
- Schema caching for performance optimization
- Change detection for schema updates

5.7 Error Handling

- Custom exception classes for different error types
- Detailed error messages for debugging
- User-friendly error responses
- Logging for system monitoring

5.8 Security Measures

- Query sanitization to prevent injection attacks
- Permission-based access control for database connections
- Encrypted storage of connection credentials
- Rate limiting for API endpoints

5.9 SQL Generation Process

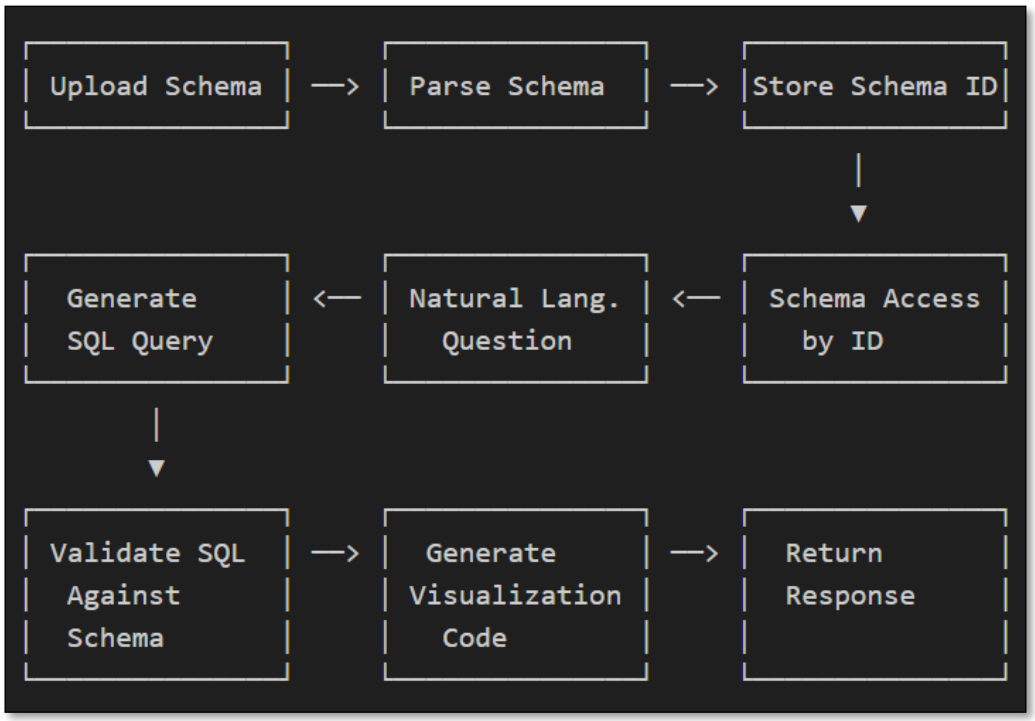


Fig 4. Flow of SQL generation

Schema Upload and Parsing

- User uploads DDL schema via /store-ddl endpoint.
- Backend parses schema with DDLSchemaParser.
- Parsed schema is stored with a unique ID .

Query Processing

- User submits a natural language question.
- Question is processed with chat history and schema.
- get_sql_query function uses BedrockLLM to generate SQL.

Validation and Visualization

- Generated SQL is validated against schema with validate_and_fix_sql.
- If errors are found, SQL is corrected automatically.
- Visualization code is generated based on query type.
- Mock data is created for visualization preview.
- Response includes original and fixed SQL, explanation, and visualization suggestions.

Database Query Execution

- User selects a generated SQL query for execution.
- Frontend sends connection parameters and query to /execute-postgres-query.
- Backend connects to the specified PostgreSQL database.
- Query is executed and results are returned.
- Table mappings can be provided to match schema tables with actual database tables.
- Auto-detection of table mappings is attempted if not explicitly provided.

6. Component Interactions

6.1 Frontend-Backend Communication

- Frontend uses api.tsx client to make requests to backend endpoints
- Authentication tokens are automatically injected into requests
- Backend processes request and returns structured JSON responses
- Error handling provides detailed information for troubleshooting

6.2 LLM Integration

- BedrockLLM class encapsulates interaction with AWS Bedrock

- Prompt engineering ensures high-quality SQL generation
- Claude 3.5 Sonnet model provides state-of-the-art SQL generation capabilities
- Error correction ensures generated SQL is valid against the provided schema

6.3 Database Interactions

- Application data is stored in PostgreSQL via SQLAlchemy ORM
- Generated SQL can be executed against user-provided databases
- Table mapping functionality bridges schema references with actual database tables
- Auto-detection of mappings simplifies user experience

7.Cost Analysis

8. Appendix

This section highlights the critical components of the implementation, both from the Frontend and Backend workflow:

Git Repository of Text-to-SQL code base: GitHub Repository

Frontend Components:

- **App.tsx**: Entry point of the application, manages routing and global providers
- **MainApp.tsx**: Primary layout component containing sidebar and main content area
- **ChatInterface.tsx**: Manages conversation flow and message display
- **Sidebar.tsx**: Navigation component that manages chat history display and selection
- **DatabaseConnectionModal**: Interface for configuring database connections
- **DdlUploadModal**: Allows users to upload database schema definitions

Backend Components:

- **FastAPI Backend**: Core server handling SQL generation and database operations
- **PostgreSQL Database**: Stores application data (users, chats, sessions)
- **Keycloak**: Authentication and authorization provider

- **AWS Bedrock**: LLM service for natural language to SQL conversion

Git Repository of Text-to-SQL code base: https://github.com/EG-A-S/eg-text-to-sql

Conclusion

The Text-To-SQL backend architecture provides a robust and secure platform for natural language to SQL conversion. The integration of FastAPI, SQLAlchemy, AWS Bedrock, and Keycloak creates a powerful system capable of understanding complex database schemas and generating accurate SQL queries from natural language questions.

The component-based design allows for easy maintenance and extension of functionality as requirements evolve

Future Scope:

a. **Deployment as standalone application**
Based on the documentation, deploying this Text-to-SQL Assistant as a standalone application would require:

- **Frontend deployment**:
  - Build the React application using Vite
  - Deploy the static assets to a web server
  - Configure environment variables for API endpoint URLs
- **Backend deployment**:
  - Deploy the FastAPI application to a server/cloud environment
  - Set up necessary environment variables (DB connection, AWS Bedrock credentials)
  - Configure CORS for frontend communication
- **Database setup**:
  - Deploy PostgreSQL database for application data storage
  - Run initial schema migrations
  - Configure secure access credentials
- **Authentication setup**:
  - Deploy Keycloak server or use a managed service
  - Configure the silent-check-sso.html for seamless authentication
  - Set up appropriate clients and roles in Keycloak
- **AWS Services**:
  - Configure AWS Bedrock access for the LLM service
  - Set up appropriate IAM permissions
- **Domain and SSL**:
  - Configure domain names (the code assumes API on subdomain api.domain.com)
  - Set up SSL certificates for secure communication
- **Networking**:

    - Configure networking to allow communication between components
    - Set up proper firewall rules and access controls

b. **Integration as Widget**

For widget integration, these items would be required:

- **Component packaging**:
    - Create a standalone component version of the chat interface
    - Package it as a web component or iframe-embeddable element
- **Authentication handling**:
    - Modify the Keycloak integration to support embedded contexts
    - Create token-passing mechanisms for host applications
- **Configuration API**:
    - Add methods to configure database connections programmatically
    - Create initialization parameters for schema information
- **Styling customization**:
    - Provide CSS variables or theming options to match host application
    - Ensure responsive design within constrained containers
- **Event communication**:
    - Implement event listeners/emitters for host application integration
    - Document the event API for third-party developers

c. **Integration as API**

For API integration, the following points are needed:

- **API documentation**:
    - Document all available endpoints (the code shows several like /store-ddl, /query, etc.)
    - Provide request/response examples for each endpoint
- **Authentication**:
    - Detail token-based authentication requirements
    - Use token refresh mechanics
- **Error handling**:
    - Document all possible error codes and responses
    - Provide troubleshooting guides
- **Client libraries**:
    - Create SDK libraries for common languages

d. **Integrating other Databases**

For supporting databases other than PostgreSQL, the documentation indicates:

- **Driver integration**:
  - Add appropriate database drivers
  - Create database-specific connection handling code
- **SQL dialect adaptation**:
  - Modify the SQL generation to account for different SQL dialects
  - Handle database-specific syntax and functions
- **Schema extraction**:
  - Implement schema extraction methods for each database type
  - Handle different metadata structures
- **Connection parameter handling**:
  - Create connection forms with appropriate fields for each database type
  - Validate connection parameters specific to each database
- **Query execution adaptations**:
  - Modify the query execution code to handle database-specific result formats
  - Account for different transaction handling
- **Error handling**:
  - Create database-specific error translations
  - Provide clear error messages for connectivity issues
- **Performance optimization**:
  - Implement database-specific query optimizations
  - Configure appropriate connection pooling